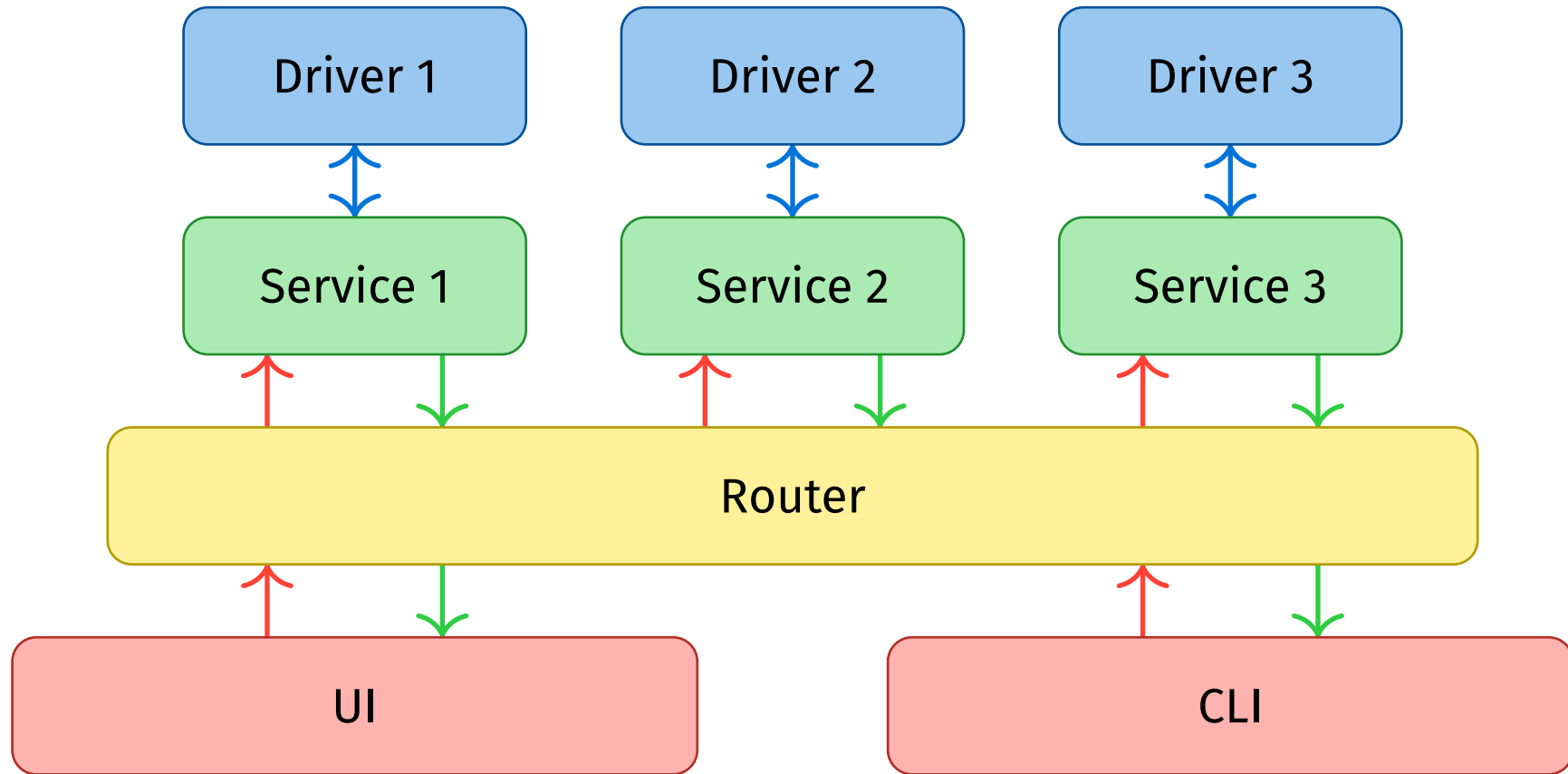


SWISS ARMY ESP

An esp32 multitool

Riccardo Segala, Ettore Beltrame

<https://github.com/RiccardoSegala04/swiss-army-esp>



Commands channels, Events channels, Function calls

- One background **Task** per Service
- Communication via async **Channels** (FIFO message queues)
- Actor-like model:
 - Services receive *Commands*
 - Services emit *Events*
- No shared mutable state

Clear boundaries between Drivers, Services, Router and UI.

Hardware-independent interfaces above, hardware-specific drivers below.

- **Safety without overhead**
 - Memory safety and no data races
 - No GC, no runtime, `no_std` friendly
- **embedded-hal ecosystem**
 - Hardware abstraction via traits
 - Portable drivers, easy mocking & testing
- **Embassy async**
 - `async/await` on bare metal
 - Natural fit for our task + channel architecture
 - No blocking, no manual state machines

This is the run function that loops in the infrared service's task

```
pub async fn run(&mut self) {  
    loop {  
        let comm = self.commands_receiver.receive().await;  
        match comm {  
            InfraredCommand::Play(sig) => {  
                self.ir.transmit(&sig).await;  
                self.send_event(InfraredEvent::SignalPlayed).await;  
            }  
            InfraredCommand::Listen => {  
                let ev = self.ir.listen().await;  
                self.send_event(ev).await;  
            }  
        }  
    }  
}
```

- **Test early, even without hardware**
 - Stub platform-specific functions
 - Use host virtual display implementation to preview UI
- **JTAG**
 - Instant, real-time debugging
 - Powerful log-level filtering



- **New peripherals can be added:**
 - CAN Interface
 - SD Card
 - ...
- **Also current peripherals could be used to add functionalities:**
 - Wi-Fi packets recording
 - MQTT Support for home automation
 - ...

The fact that the code is modular and follows a service-based approach makes adding new Views, Devices and Services easy